ESGI · ASSEMBLEUR X86

TP 1 — Environnement de travail et premier programme

Guide étudiant — pas à pas

Enseignant : **Youssef YOUNES**

Année universitaire 2026-2027

Polys générés à l'aide d'une IA — contenu relu et validé par l'enseignant.

DÉPÔT DU COURS
`github.com/youssef-esgi/asm-esgi`

Exercices, tests, instructions d'installation et ce guide en PDF. Pour le récupérer :

```
git clone https://github.com/youssef-esgi/asm-esgi.git
cd asm-esgi
```

Les commandes de ce guide se copient plus facilement depuis le fichier `README.md` du dépôt que depuis ce PDF.

1. Ce dont vous avez besoin

Le cours utilise un seul environnement pour tout le monde, quel que soit votre ordinateur : un conteneur Docker Linux contenant l'assembleur `nasm`, le compilateur `gcc` 32 bits, le débogueur `gdb` et l'outil `make`. Vous éditez vos fichiers sur votre machine avec votre éditeur habituel ; le conteneur ne sert qu'à compiler, exécuter et déboguer.

Trois façons d'y accéder, choisissez-en une :

Option	Pour qui	Avantages / limites
A. GitHub Codespaces	Tout le monde, en particulier si votre PC refuse Docker ou si vous êtes sur un poste de l'école	Rien à installer, VS Code dans le navigateur. Quota : 60 h/mois (90 h avec le GitHub Student Pack). Nécessite Internet.
B. Docker sur Mac	Mac Intel ou Apple Silicon (M1 à M4)	Fonctionne hors ligne une fois l'image téléchargée. Docker Desktop consomme 2 à 4 Go de RAM.
C. Docker sur Windows	PC Windows 10/11 64 bits avec 8 Go de RAM	Idem Mac, mais l'installation demande WSL 2 et parfois un réglage du BIOS. Suivre le document `INSTALL-WINDOWS.md`.

> **À RETENIR**
> L'installation se fait en classe, au début de la première séance, encadrée par l'enseignant. N'installez rien seul chez vous : en cas de blocage, vous perdriez du temps sans aide. Venez avec votre ordinateur portable et son chargeur.
>
> Une seule chose à préparer avant de venir : créer un compte Docker Hub gratuit, expliqué en 2.4. C'est ce qui évite un blocage collectif au moment du téléchargement.

2. Installation

2.1 Option A — GitHub Codespaces

Étape 1 — Créer un compte GitHub

Sur github.com, avec votre adresse étudiante de préférence. Demandez ensuite le GitHub Student Developer Pack : il augmente le quota Codespaces.

Étape 2 — Ouvrir le dépôt du cours

Rendez-vous sur github.com/youssef-esgi/asm-esgi. Cliquez sur le bouton vert Code, onglet Codespaces, puis Create codespace on main.

Étape 3 — Attendre l'ouverture de VS Code

La première ouverture prend une à deux minutes : GitHub télécharge l'image du cours. Vous arrivez dans VS Code, dans votre navigateur, avec un terminal en bas. Si le terminal n'est pas visible : menu Terminal → New Terminal.

Étape 4 — Vérifier

Dans le terminal :

```
cd exercices/ex01-somme-tableau
make check
```

La dernière ligne doit être `Environnement prêt`. Passez à la section 3.

> **À RETENIR**
> Arrêtez votre codespace quand vous avez fini (menu en bas à gauche → Stop Current Codespace), sinon il continue de consommer votre quota. Vos fichiers sont conservés.

2.2 Option B — Docker sur Mac

Étape 1 — Installer Docker Desktop

Téléchargez Docker Desktop en choisissant Apple Silicon (Mac M1, M2, M3, M4) ou Intel selon votre Mac. Pour savoir lequel vous avez : menu Pomme → À propos de ce Mac, ligne Puce ou Processeur. Ouvrez le fichier `.dmg`, glissez Docker dans Applications, lancez-le et acceptez la licence. Un compte Docker Hub n'est pas obligatoire, mais recommandé (voir 2.4).

Étape 2 — Attendre que Docker soit démarré

L'icône de baleine dans la barre de menus doit cesser de s'animer. Dans l'application, le coin inférieur gauche affiche Engine running.

Étape 3 — Récupérer le dépôt

Ouvrez le Terminal (Applications → Utilitaires, ou Spotlight puis « Terminal »), puis :

```
cd ~/Documents
git clone https://github.com/youssef-esgi/asm-esgi.git assembleur
cd assembleur
```

Si `git` n'est pas installé, macOS propose d'installer les outils en ligne de commande : acceptez, puis relancez la commande.

Étape 4 — Lancer le conteneur

```
./dev.sh
```

La première fois, l'image du cours est téléchargée. Vous obtenez ensuite une invite Linux `root@...:/work#`. Vous êtes dans le conteneur ; le dossier `/work` est votre dossier `assembleur`, en direct.

Étape 5 — Vérifier

```
cd exercices/ex01-somme-tableau
make check
```

La dernière ligne doit être `Environnement prêt`. Tapez `exit` pour sortir du conteneur.

2.3 Option C — Docker sur Windows

Le pas à pas complet avec les captures d'écran et le dépannage se trouve dans le fichier `INSTALL-WINDOWS.md` du dépôt. En résumé :

Étape 1 — Activer WSL 2

PowerShell en administrateur : `wsl --install`, puis redémarrer.

Étape 2 — Installer Docker Desktop

Depuis le site Docker, version Windows AMD64, en gardant l'option Use WSL 2. Redémarrer si demandé, lancer Docker Desktop, attendre Engine running.

Étape 3 — Installer Git pour Windows

Depuis git-scm.com, options par défaut. Cela installe Git Bash, le terminal à utiliser pour la suite.

Étape 4 — Récupérer le dépôt et lancer

Dans Git Bash :

```
cd ~/Documents
git clone https://github.com/youssef-esgi/asm-esgi.git assembleur
cd assembleur
./dev.sh
cd exercices/ex01-somme-tableau
make check
```

Si `./dev.sh` répond the input device is not a TTY, vous êtes dans PowerShell : utilisez Git Bash. Si votre PC refuse d'installer Docker (virtualisation désactivée, pas de droits administrateur), passez à l'option A.

2.4 Compte Docker Hub (options B et C)

Sans compte, Docker Hub limite les téléchargements à 100 par 6 heures par adresse IP. En salle de TD, toute la classe partage la même adresse : sans compte, le téléchargement échoue pour une partie du groupe.

Créez ce compte gratuit avant la séance sur hub.docker.com. C'est la seule préparation demandée. En classe, vous taperez `docker login` dans votre terminal, une fois Docker Desktop installé.

3. Comprendre le kit

3.1 Organisation du dépôt

```
Dockerfile              recette de l'image Docker (vous n'y touchez pas)
.devcontainer/          configuration VS Code / Codespaces
dev.sh                  lance le conteneur Docker sur votre machine
common.mk               règles make partagées par tous les exercices
tools/run-tests.sh      compare la sortie du programme aux résultats attendus
tools/check-env.sh      diagnostic (make check)
exercices/
  ex01-somme-tableau/
    Makefile            4 lignes : nom du programme, fichiers .asm et .c
    main.c              FOURNI : lit les données, appelle vos fonctions, affiche
    somme.asm           VOTRE TRAVAIL : les fonctions en assembleur
    tests/              01-simple.in, 01-simple.out, ...
    README.md           énoncé court
```

3.2 Les commandes

Toutes se lancent dans le dossier de l'exercice, à l'intérieur du conteneur :

Commande	Effet
`make`	Assemble les `.asm` avec `nasm`, compile les `.c` avec `gcc`, lie le tout en un exécutable 32 bits.
`make run`	Compile puis exécute. Vous tapez les données au clavier.
`make test`	Compile puis exécute le programme sur chaque fichier `tests/*.in` et compare la sortie au fichier `.out` correspondant. Affiche `PASS` ou `FAIL` avec les différences.
`make debug`	Compile puis ouvre `gdb`, arrêté à l'entrée de la première fonction de votre `.asm`, avec le premier fichier `tests/*.in` comme entrée. Variante : `make debug BREAK=somme_tableau IN=tests/03-negatifs.in`.
`make check`	Vérifie que l'environnement est complet et fonctionne.
`make clean`	Supprime les fichiers générés (`.o` et l'exécutable).

3.3 Le principe : C pour les entrées-sorties, assembleur pour la logique

Lire un nombre au clavier ou afficher un résultat en assembleur pur demande une cinquantaine de lignes sans intérêt pédagogique au début. Nous les confions au C : `main.c` lit les données avec `scanf`, appelle vos fonctions écrites en assembleur, et affiche le résultat avec `printf`. Vous vous concentrez sur les registres, la mémoire et la pile.

Pour que le C puisse appeler votre fonction, les deux doivent respecter le même contrat : la convention d'appel cdecl, détaillée en 4.2.

4. Exercice 1 pas à pas — Somme d'un tableau

Énoncé. Écrire dans `somme.asm` la fonction `int somme_tableau(const int *t, int n)` qui retourne la somme des `n` entiers de 32 bits du tableau `t`. Le tableau peut être vide (`n = 0`) : la somme vaut alors 0.

4.1 Lire `main.c`

Ouvrez `exercices/ex01-somme-tableau/main.c`. Vous n'avez pas à le modifier, mais il faut comprendre ce qu'il fait :

```c
int addition(int a, int b);             /* exemple fourni  */
int somme_tableau(const int *t, int n); /* à écrire        */

int main(void)
{
    int n;
    int t[100];
    scanf("%d", &n);                    /* nombre d'éléments   */
    for (int i = 0; i < n; i++)
        scanf("%d", &t[i]);             /* les éléments        */

    printf("addition(2, 3) = %d\n", addition(2, 3));
    printf("somme = %d\n", somme_tableau(t, n));
    return 0;
}
```

Les deux premières lignes sont des déclarations : elles disent au compilateur C que ces fonctions existent quelque part, avec ces types d'arguments. Le « quelque part », c'est votre fichier `somme.asm`, et c'est l'éditeur de liens (`gcc` à la dernière étape de `make`) qui fait le raccord grâce aux lignes `global addition` et `global somme_tableau` de votre fichier.

Format d'entrée : d'abord `n`, puis `n` entiers séparés par des espaces ou des sauts de ligne. Exemple, contenu de `tests/01-simple.in` :

```
3
1 2 3
```

Sortie attendue (`tests/01-simple.out`) :

```
addition(2, 3) = 5
somme = 6
```

4.2 La convention d'appel cdecl

Quand `main` exécute `somme_tableau(t, n)`, le compilateur C génère ceci :

```
push n              ; dernier argument empilé en premier
push t              ; ... puis le premier argument
call somme_tableau  ; empile l'adresse de retour et saute
add  esp, 8         ; l'appelant nettoie la pile (2 arguments × 4 octets)
```

À l'entrée de votre fonction, la pile ressemble donc à ceci (elle descend : chaque `push` retire 4 à `esp`) :

Adresse	Contenu	Comment y accéder
`esp+8`	`n` (2e argument)	`[esp+8]` à l'entrée
`esp+4`	`t` (1er argument)	`[esp+4]` à l'entrée
`esp`	adresse de retour	posée par `call`

Le problème : dès que votre fonction fait un `push`, `esp` change et les décalages aussi. D'où le prologue standard, qui fixe un point de repère `ebp` qui ne bouge plus :

```
push ebp            ; sauvegarde l'ancien ebp (celui de main)
mov  ebp, esp       ; ebp = sommet de pile actuel : notre repère
```

Après ces deux lignes, la pile est :

Adresse	Contenu	Comment y accéder
`ebp+12`	`n`	`[ebp+12]`
`ebp+8`	`t`	`[ebp+8]`
`ebp+4`	adresse de retour	ne pas toucher
`ebp`	ancien `ebp`	restauré par `pop ebp`

Et l'épilogue remet tout en place avant de rendre la main :

```
pop  ebp            ; restaure l'ebp de main
ret                 ; dépile l'adresse de retour et y saute
```

À RETENIR
Le contrat cdecl en quatre règles :
1. Arguments sur la pile, à `[ebp+8]`, `[ebp+12]`, `[ebp+16]` … après le prologue.
2. La valeur de retour est dans `eax`.
3. `eax`, `ecx`, `edx` sont libres : vous pouvez les écraser.
4. `ebx`, `esi`, `edi`, `ebp` doivent avoir la même valeur en sortie qu'en entrée : si vous les utilisez, faites `push` au début et `pop` à la fin, dans l'ordre inverse.

4.3 Lire `addition`, ligne par ligne

```
addition:
    push ebp                ; prologue : sauvegarde ebp
    mov  ebp, esp           ;            ebp = repère fixe

    mov  eax, [ebp+8]       ; eax = a      (1er argument)
    add  eax, [ebp+12]      ; eax = a + b  (2e argument)

    pop  ebp                ; épilogue : restaure ebp
    ret                     ;            retour à l'appelant
```

Points à observer :

- Les crochets `[...]` signifient le contenu de la mémoire à cette adresse. Sans crochets, `mov eax, ebp` copierait l'adresse elle-même.
- `add eax, [ebp+12]` lit en mémoire et additionne en une seule instruction. En x86, un seul opérande peut être en mémoire par instruction : `add [ebp+8], [ebp+12]` est interdit.
- Le résultat est laissé dans `eax` : c'est là que `main` ira le chercher.

4.4 Compiler et tester le squelette

Avant d'écrire quoi que ce soit, lancez `make test` :

```
$ make test
nasm -f elf32 -g -F dwarf -o somme.o somme.asm
gcc -m32 -g -O0 -Wall -Wextra -fno-pie -c -o main.o main.c
gcc -m32 -static -no-pie -Wl,-z,noexecstack -o somme somme.o main.o
FAIL 01-simple
     2c2
     < somme = 0
     ---
     > somme = 6
PASS 02-vide
FAIL 03-negatifs
     ...
-----
1 réussi(s), 3 échoué(s)
```

Trois lignes de compilation, puis les tests. Le squelette fourni fait `mov eax, 0` : il retourne toujours 0, donc seul le test du tableau vide passe. Les lignes `<` montrent votre sortie, les lignes `>` la sortie attendue.

4.5 Écrire `somme_tableau`

Étape 1 — Écrire l'algorithme en français

```
somme = 0
adresse = t
compteur = n
tant que compteur != 0 :
    somme = somme + (entier à l'adresse)
    adresse = adresse + 4        (un entier = 4 octets)
    compteur = compteur - 1
retourner somme
```

Étape 2 — Attribuer un registre à chaque variable

Variable	Registre	Pourquoi
somme	`eax`	C'est la valeur de retour : autant l'accumuler directement dedans.
compteur	`ecx`	Registre « compteur » par convention, et libre (pas à sauvegarder).
adresse	`esi`	Registre « source index », conçu pour parcourir la mémoire. Il est callee-saved : à sauvegarder.

Étape 3 — Écrire le prologue et charger les arguments

```
somme_tableau:
    push ebp
    mov  ebp, esp
    push esi                ; esi doit être préservé : on le sauvegarde

    mov  esi, [ebp+8]       ; esi = t   (adresse du 1er élément)
    mov  ecx, [ebp+12]      ; ecx = n
    xor  eax, eax           ; eax = 0   (idiome : plus court que mov eax, 0)
```

Attention : le `push esi` vient après `mov ebp, esp`, sinon `ebp` ne pointerait plus au bon endroit et `[ebp+8]` serait faux.

Étape 4 — Traiter le cas n = 0

Si l'on entre dans la boucle avec `ecx = 0`, le `dec ecx` donnera `-1` puis la boucle tournera quatre milliards de fois. Il faut tester avant :

```
    test ecx, ecx           ; positionne ZF si ecx == 0, sans modifier ecx
    jz   .fin
```

Étape 5 — La boucle

```
.boucle:
    add  eax, [esi]         ; eax += *esi
    add  esi, 4             ; élément suivant
    dec  ecx                ; compteur--, positionne ZF si résultat nul
    jnz  .boucle            ; si ecx != 0, recommencer
```

Les étiquettes commençant par un point (`.boucle`, `.fin`) sont locales à la fonction précédente : vous pouvez réutiliser `.boucle` dans une autre fonction sans conflit.

Étape 6 — L'épilogue, dans l'ordre inverse du prologue

```
.fin:
    pop  esi                ; restaure esi
    pop  ebp                ; restaure ebp
    ret
```

Étape 7 — Tester

```
$ make test
...
PASS 01-simple
PASS 02-vide
PASS 03-negatifs
PASS 04-grands
-----
4 réussi(s), 0 échoué(s)
```

4.6 Déboguer avec gdb, pas à pas

Le test automatique dit si c'est faux, gdb dit pourquoi. Session type sur le test `01-simple` :

Étape 1 — Lancer

```
make debug BREAK=somme_tableau
```

Le programme démarre avec `tests/01-simple.in` en entrée et s'arrête à la première instruction de `somme_tableau`. L'écran est divisé : les registres en haut, le code source au milieu, la ligne de commande gdb en bas. La ligne surlignée est la prochaine instruction à exécuter.

Étape 2 — Regarder la pile à l'entrée

```
(gdb) x/3xw $esp
```

Affiche trois mots de 4 octets à partir de `esp` : l'adresse de retour, puis `t`, puis `n`. Vous devez voir `3` en troisième position.

Étape 3 — Avancer instruction par instruction

```
(gdb) si
```

Tapez `si` (step instruction) trois fois pour passer le prologue et le `push esi`. Observez `esp` diminuer de 4 à chaque `push` dans la fenêtre des registres, et `ebp` prendre la valeur de `esp` après le `mov`.

Étape 4 — Vérifier les arguments chargés

Après les deux `mov` :

```
(gdb) p/x $esi          affiche esi en hexadécimal : l'adresse du tableau
(gdb) p $ecx            affiche ecx en décimal : doit valoir 3
(gdb) x/3dw $esi        les 3 entiers du tableau : 1 2 3
```

Étape 5 — Suivre la boucle

Continuez avec `si` et notez à chaque tour :

Tour	`eax`	`ecx`	`esi` par rapport au départ
avant	0	3	+0
1	1	2	+4
2	3	1	+8
3	6	0	+12

Après le `dec ecx` du dernier tour, la fenêtre des registres montre `ZF` positionné dans `eflags` (affiché `[ ... ZF ... ]`) : le `jnz` ne saute pas, on sort.

Étape 6 — Terminer

```
(gdb) finish        exécute jusqu'au retour de la fonction, dans main.c
(gdb) p $eax        la valeur retournée (finish ne l'affiche pas pour une fonction
en assembleur)
(gdb) c             continue jusqu'à la fin du programme
(gdb) q             quitte gdb
```

À RETENIR
Commandes gdb à connaître :

`si`	**une instruction (entre dans les `call`)**
`ni`	une instruction (passe par-dessus les `call`)
`c`	continue jusqu'au prochain point d'arrêt
`finish`	termine la fonction courante et revient dans `main.c` ; puis `p $eax` pour la valeur retournée
`p $eax`, `p/x $eax`	valeur d'un registre, en décimal ou en hexadécimal
`x/4dw $esp`	4 mots de 4 octets à l'adresse `esp`, en décimal
`x/8xb $esi`	8 octets à l'adresse `esi`, en hexadécimal
`info registers`	tous les registres et les drapeaux
`break addition`	ajoute un point d'arrêt
`ctrl-L`	redessine l'écran s'il est brouillé
`q`	quitte

4.7 gdb à la main, sans `make debug`

`make debug` enchaîne des commandes que vous pouvez taper vous-même. Il est utile de savoir le faire.

Lire le code sans rien exécuter. L'exécutable contient les symboles et le lien vers votre source. Aucun `run`, aucun point d'arrêt nécessaire :

```
gdb ./somme                      # sur Mac Apple Silicon : gdb-multiarch ./somme
(gdb) list somme_tableau         # votre source .asm, avec les numéros de ligne
(gdb) disassemble somme_tableau  # le code machine désassemblé, syntaxe Intel
(gdb) x/12i addition             # 12 instructions à partir de addition
(gdb) info functions somme       # cherche les symboles contenant "somme"
(gdb) q
```

La syntaxe Intel est déjà réglée par le fichier `.gdbinit` de l'image : inutile de taper `set disassembly-flavor intel`. Sans gdb : `objdump -d -M intel somme`.

Exécuter sur PC, Codespaces ou Linux (x86_64). gdb lance le programme lui-même :

```
gdb ./somme
(gdb) break somme_tableau
(gdb) run < tests/01-simple.in
(gdb) layout regs
```

Exécuter sur Mac Apple Silicon (arm64). Le programme est en x86, votre processeur ne l'exécute pas directement : c'est `qemu-i386` qui le lance, et gdb s'y connecte par le réseau. Il faut deux terminaux dans le conteneur (ou un seul, avec `&` derrière la première commande) :

```
# Terminal 1 : qemu lance le programme et attend gdb sur le port 1234
qemu-i386 -g 1234 ./somme < tests/01-simple.in
```

```
# Terminal 2 : gdb se connecte à qemu
gdb-multiarch ./somme
(gdb) set architecture i386
(gdb) target remote localhost:1234
(gdb) break somme_tableau
(gdb) continue                  # PAS run : le programme est déjà lancé par qemu
(gdb) layout regs
```

Pour ouvrir un second terminal dans le même conteneur : depuis votre Mac, `docker ps` pour lire le nom du conteneur, puis `docker exec -it <nom> bash`.

À RETENIR
Sur Mac, `run` dans gdb échoue toujours : gdb essaierait d'exécuter un programme x86 sur un processeur ARM. Le programme est lancé par qemu, gdb ne fait que s'y attacher avec `target remote`, puis on avance avec `continue`.

5. Erreurs fréquentes

Message ou symptôme	Cause et solution
`make: *** No rule to make target`	Vous n'êtes pas dans le dossier de l'exercice. `cd exercices/ex01-somme-tableau`.
`nasm: error: symbol 'somme_tableau' redefined`	Deux étiquettes avec le même nom. Utilisez des étiquettes locales `.boucle`.
`undefined reference to 'somme_tableau'`	La ligne `global somme_tableau` manque, ou l'étiquette est mal orthographiée (l'assembleur distingue majuscules et minuscules).
`error: invalid combination of opcode and operands`	Deux opérandes mémoire, ou tailles incompatibles (`mov eax, bl`). Passez par un registre.
`FAIL 02-vide : délai dépassé`	Boucle infinie : le cas `n = 0` n'est pas géré avant la boucle.
`Segmentation fault`	Lecture à une adresse invalide (confusion adresse / valeur, `pop` dans le mauvais ordre) ou `ret` avec une pile déséquilibrée. Utilisez gdb : le crash s'affiche avec l'instruction fautive.
Résultat correct pour `01-simple` mais faux pour `03-negatifs`	Vous utilisez un registre 8 ou 16 bits (`al`, `ax`) au lieu de `eax`, ou une instruction non signée.
`the input device is not a TTY` (Windows)	Lancez `./dev.sh` depuis Git Bash, pas depuis PowerShell.
`toomanyrequests` au premier `./dev.sh`	Limite Docker Hub. Faites `docker login`, ou tirez l'image depuis chez vous.

6. Pour aller plus loin

1. Écrire `int maximum(const int *t, int n)` dans le même fichier, ajouter sa déclaration et son appel dans `main.c`, et un test `tests/05-max.in` / `.out`. Attention : que retourner si `n = 0` ?
2. Réécrire `somme_tableau` avec l'adressage indexé `[edx + ecx*4]` au lieu d'avancer `esi`. Quel registre callee-saved n'est plus nécessaire ?
3. Dans gdb, mettre un point d'arrêt après l'appel dans `main` (`break main.c:23`) et vérifier que `esi` a la même valeur qu'avant l'appel. Supprimez le `push esi` / `pop esi` et recommencez : que se passe-t-il ? Pourquoi les tests passent-ils quand même ?